Experiment 5

Aim: Case Study on Hyperledger.

Blockchain Technology: Healthcare Record Sharing System using Hyperledger Fabric

Steps:

Follow the next steps to build your Private Blockchain.

- Install Geth
- Define Genesis Block
- Start the first blockchain node
- Start the second blockchain node
- Create the peer-to-peer connection
- Mining Blocks and Create Transactions

Installing Geth (Go Ethereum)

Geth is a CLI with some resources to connect you to Ethereum network. It will be used to start our private network in local environment.
To install Geth in Ubuntu/Debian follow the following steps:

- sudo add-apt-repository -y ppa:ethereum/ethereum
- sudo apt-get update
- sudo apt-get install ethereum

Check if geth was installed successfully with the following command.

- geth -h

This shows all available commands using geth.

The Genesis Block

All blockchains start with the genesis block. This block define the initial configuration to a blockchain. The configuration to genesis block is defined in *genesis.json* file.
So, let's create a folder to start the private network and create the *genesis.json* file.

- mkdir my-blockchain
- cd my-blockchain
- touch genesis.json

Add the following content to *genesis.json* file:

```
{
 "config": {
  "chainId": 1234,
  "homesteadBlock": 0,
  "eip150Block": 0,
  "eip155Block": 0,
  "eip158Block": 0,
  "byzantiumBlock": 0,
  "constantinopleBlock": 0,
```

```
  "petersburgBlock": 0,
  "ethash": {}
 },
 "difficulty": "4",
 "gasLimit": "8000000",
 "alloc": {}
}
```

Explanation:

- chainId: An integer greater than zero that defines a unique id for your private network. The main network has ID 1. If you supply your own custom network ID which is different than the main network, your nodes will not connect to other nodes and form a private network.. You can find a community-run registry of Ethereum networks at https://chainid.network.
- homesteadBlock: Homestead is the first official stable version of the Ethereum protocol. If you plan to use this release, the attribute should be set to *0*.
- difficulty: Determines the difficulty of generating blocks.
- gasLimit: Indicates the current network-wide gas consumption limit per unit. Gas is the *fuel* that is used to pay transaction fees on the Ethereum network.
- alloc: Is used to start some accounts with positive balance in the network. At this case we not defined any account.
- The other attributes refer to the protocols and attributes that will be used on the network, we set the default values for the ethereum network.

Start Database (The Bootnode)

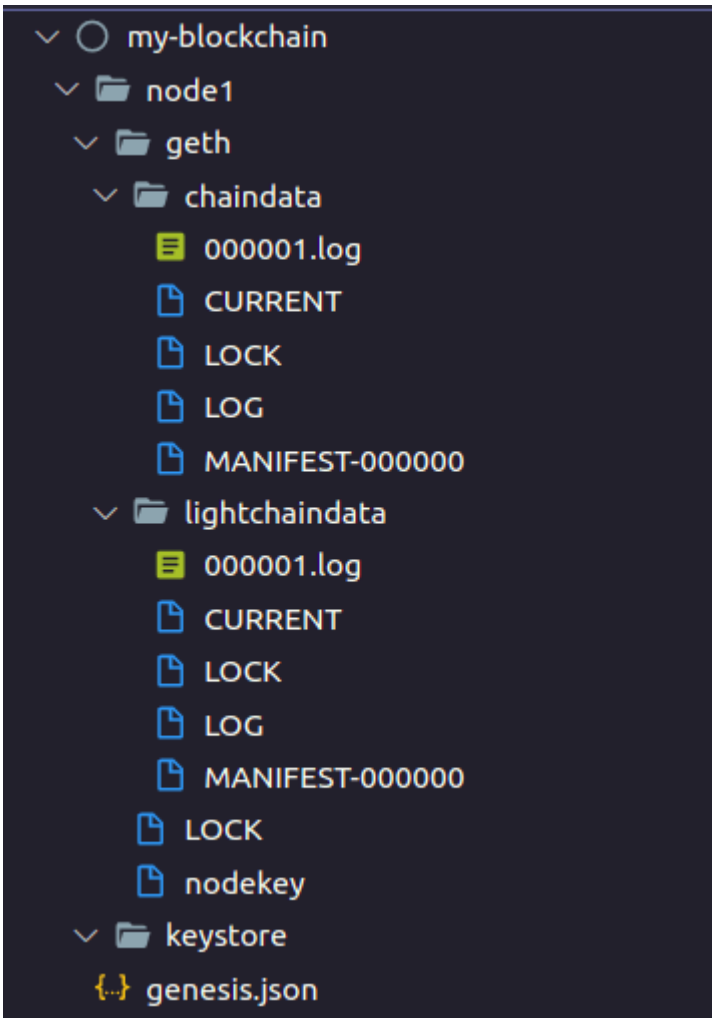
The bootnode is the first node created when the blockchain is started. To start the database to first node execute:

- geth init --datadir node1 genesis.json

The folder *node1* will be created with the database to bootnode.
The terminal result must be something like this:

```
jefferson in ~/projects/my-blockchain
➜ geth init --datadir node1 genesis.json
INFO [03-17|16:10:25.866] Maximum peer count                       ETH=50 LES=0 total=50
INFO [03-17|16:10:25.866] Smartcard socket not found, disabling    err="stat /run/pcscd/pcscd.comm: no such file or directory"
INFO [03-17|16:10:25.867] Set global gas cap                       cap=50,000,000
INFO [03-17|16:10:25.867] Allocated cache and file handles         database=/home/jefferson/projects/my-blockchain/node1/geth/chaindata cache=16.
00MiB handles=16
INFO [03-17|16:10:26.322] Writing custom genesis block
INFO [03-17|16:10:26.323] Persisted trie from memory database      nodes=0 size=0.00B time="32.042µs" gcnodes=0 gcsize=0.00B gctime=0s livenodes=
1 livesize=0.00B
INFO [03-17|16:10:26.324] Successfully wrote genesis state         database=chaindata hash=ac4d9a..2ca582
INFO [03-17|16:10:26.325] Allocated cache and file handles         database=/home/jefferson/projects/my-blockchain/node1/geth/lightchaindata cach
e=16.00MiB handles=16
INFO [03-17|16:10:26.812] Writing custom genesis block
INFO [03-17|16:10:26.813] Persisted trie from memory database      nodes=0 size=0.00B time="22.667µs" gcnodes=0 gcsize=0.00B gctime=0s livenodes=
1 livesize=0.00B
INFO [03-17|16:10:26.814] Successfully wrote genesis state         database=lightchaindata hash=ac4d9a..2ca582
```

The directory result should be something like this:

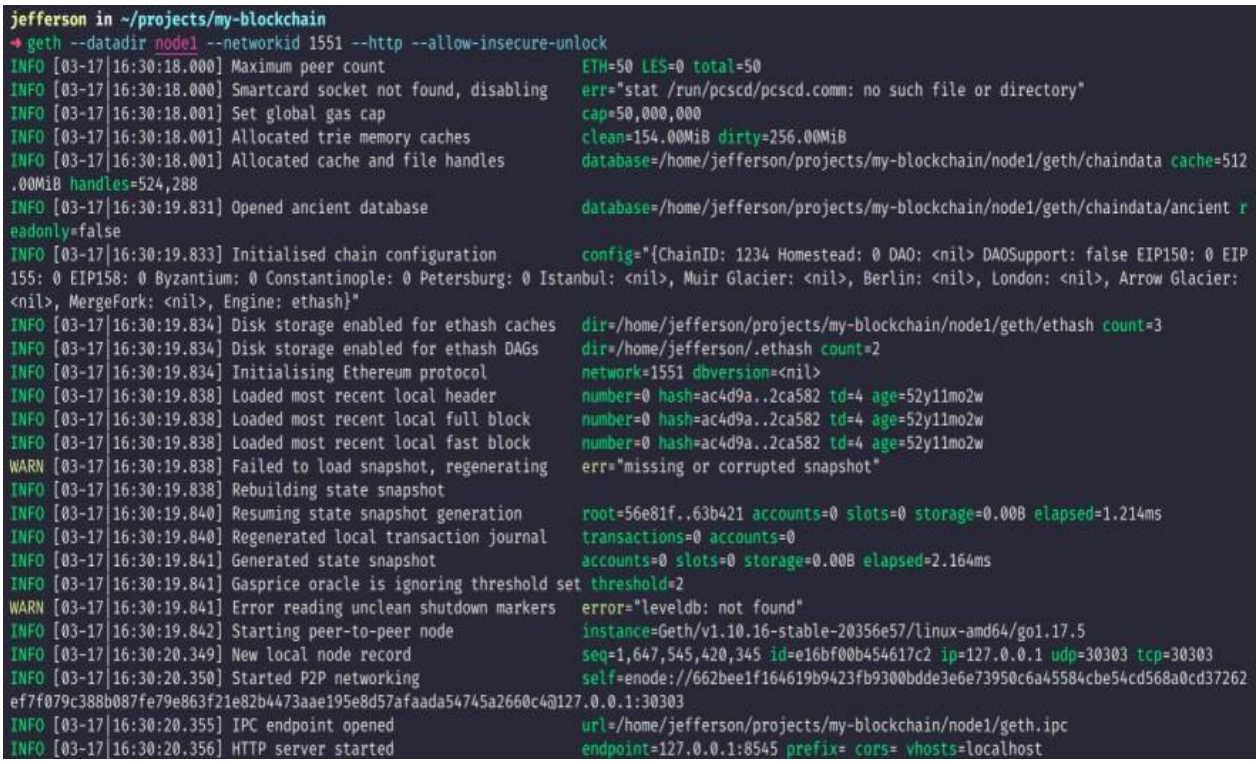


Start Node

- geth --datadir node1 --networkid 1234 --http --allow-insecure-unlock --nodiscover

This command will be start the node in private network with id 1234.
The flag --*http* is used to enable Web App Access
The flag --*allow-insecure-unlock* is used to permit execute transfers without a web application, The flag --*nodiscover* is used to prevent node from trying to connect to others automatically.

The terminal result must be something like this:



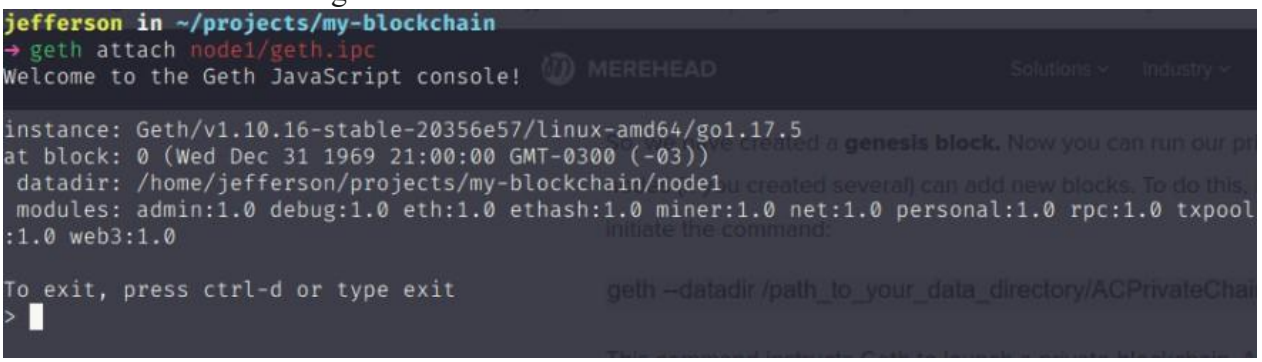
The blockchain in the private network is running.

The following steps can be used to test it:

Open other terminal window in the same folder *my-blockchain* and execute the following command:

- geth attach node1/geth.ipc

This opens an interactive javascript terminal to execute some tasks to this node. The terminal result should be something like this:



To see node informations execute the following command in this javascript terminal opened:

- admin.nodeInfo

This command show all informations about this node, and you can verify the node is active. Keep this window open for some more tests ahead.



Adding member peers

Let's start the second node to our blockchain. To do this execute the following commands:

- geth init --datadir node2 genesis.json
- geth --datadir node2 --networkid 1234 --port 30304

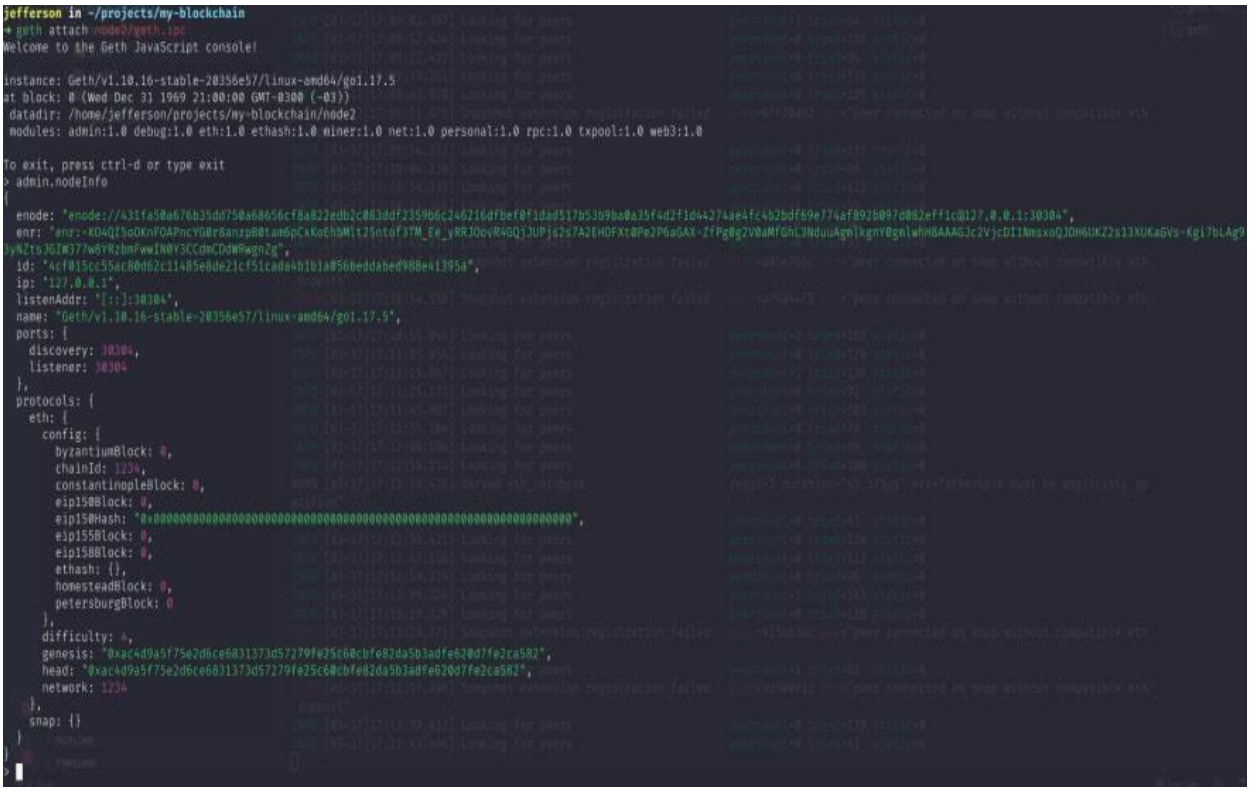
The commands are very similar to the previous ones with some minor differences:

- The folder to database in this case is *node2*.
- The flags *--http* and *--allow-insecure-unlock* are not necessary.
- The flag *--port* it's necessary to differentiate this node from the previous one. The first node is running on default port 30303.

Open a new javascript terminal to second node created and verify the informations executing:

- geth attach node2/geth.ipc
- admin.nodeInfo

The result should be something like this:



Connecting Peers

Now, let's create the peer-to-peer connection between the two nodes in blockchain. Copy the enode from the *nodeInfo* of the second node, then execute the following command in javascript terminal of the first node:

- admin.addPeer("enode://431fa50a676b35dd750a68656cf8a822edb2c083ddf2359b6c246216dfbef0f1dad517b53b9ba0a35f4d2f1d44274ae4fc4b2bdf69e774af892b097d082eff1c@127.0.0.1:30304?discport=0")

And to see all peers connected execute to both nodes the following command:

- admin.peers

Any node must be have one peer connected.
Note the 30304 port referencing the second node in network. The result should be something like this:

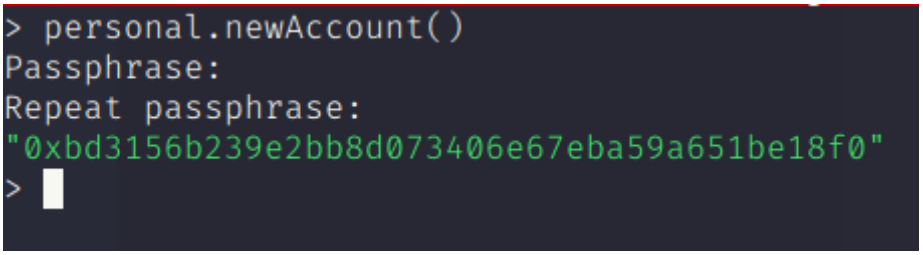


Done!! Your blockchain is created with multiple nodes runing and connected. Let's create accounts and start mining to see data updated in both nodes.

Mining

To mine blocks it's necessary have a base account. Let's do this. In javascript terminal of the first node execute the command to create a new account and define the password required to this account, the public address from created account should be presented.

- personal.newAccount()



You can see account balance executing:

- eth.getBalance("0xbd3156b239e2bb8d073406e67eba59a651be18f0")

Now, you can start the mining in the first node. Before that run the command to see the current height of the blocks, must be 0, because no blocks have been mined yet:

- eth.blockNumber

Start the mining, and stop after some seconds to see updated block.

- miner.start() // Start Mining
- miner.stop() // Stop Mining

Now Verify again the blockNumber and balance of account created. Some blocks must have been created and values must have been earned as a reward.

```
> miner.start()
null
> miner.stop()
null
> eth.blockNumber
57
> eth.getBalance("0xbd3156b239e2bb8d073406e67eba59a651be18f0")
11400000000000000000
>
```

You can verify the blockNumber also in javascript terminal of the second node after some seconds, the value must be the same presented in the first node. This means that in fact the nodes are connected and are updated.

Blockchain Structure

